Résumé de cours : Formes et sources de parallélisme

Évaluation des performances

1 Formes de parallélisme

Implicite Le parallélisme est extrait automatiquement (compilateur, matériel : pipeline, exécution superscalaire, etc.). Le programmeur n'intervient pas.

Explicite Le programmeur parallélise lui-même (manuellement) : threads, MPI, OpenMP, etc.

Deux approches de parallélisation.

1. **Séquentiel → parallèle** : on part d'un programme existant et on l'adapte.
2. **Parallèle directement** : on conçoit le programme parallèle dès le départ. C'est facile sur des bases de code simples, mais plus difficile quand le code est complexe.

2 Sources de parallélisme

Question de base : *que regarde-t-on dans un programme pour dire s'il peut être parallélisé ou non ?* Il existe trois sources principales.

2.1 Parallélisme de données

- Même tâche appliquée à beaucoup de données : travail répétitif, de faible complexité, facile à découper.
- Typique des **données régulières** : tableaux, matrices, listes identiques.
- En mémoire distribuée : on applique en parallèle le même calcul à des données différentes.
- La difficulté principale est la **distribution** des tâches/données.
- Des tâches différentes $\not\Rightarrow$ parallélisme de données.

2.2 Parallélisme de tâches

- On applique des calculs *différents* sur des données *différentes*.
- Beaucoup de tâches différentes : on ne peut pas les répartir au hasard, donc on n'utilise pas les techniques du parallélisme de données.
- Beaucoup de tâches et beaucoup de données : on **étudie les dépendances** et on choisit la meilleure approche.
- Objectif : minimiser le temps d'exécution et les ressources de calcul.

2.3 Parallélisme de flux (pipeline)

- Correspond au **travail à la chaîne** : chaque processeur est un spécialiste qui réalise une étape.
- Chaque donnée subit une séquence de traitements réalisés en mode pipeline, en exploitant la régularité des données.
- Moins courant que les deux autres formes.
- Exemple : une chaîne de lavage de voiture (laver, rincer, sécher, …) où chaque poste traite une voiture différente en même temps.

Parallélisme mixte

Lorsqu'on combine plus d'un mode (données, tâches, flux), on parle de **parallélisme mixte**.

3 Parallélisme de tâches : exemple

On considère quatre tâches, de durée 1 unité de temps (u.t.) chacune :

$$t_1 : a \leftarrow a+2, \quad t_2 : b \leftarrow b-4, \quad t_3 : d \leftarrow d+9, \quad t_4 : c \leftarrow a \times b - d.$$

La tâche t_4 dépend de t_1, t_2 et t_3.

Version 1 : $t_1 \parallel t_2 \parallel t_3 \Rightarrow t_4$. Séquentiel : $T_1 = 4$. Parallèle sur $p = 3$ processeurs : $T_3 = 2$.

$$S_3 = \frac{T_1}{T_3} = \frac{4}{2} = 2, \qquad E_3 = \frac{S_3}{p} = \frac{2}{3} \approx 66\,\%.$$

On accélère le programme 2 fois, mais un processeur est inactif pendant la 2e unité de temps.

Version 2 : $t_1 \parallel t_2 \Rightarrow t_3 \parallel t_4$. Ici on regroupe les calculs pour n'utiliser que $p = 2$ processeurs : $T_1 = 4$, $T_2 = 2$.

$$S_2 = \frac{4}{2} = 2, \qquad E_2 = \frac{S_2}{p} = \frac{2}{2} = 1.$$

(Cette version suppose que t_4 est réécrit de manière à ne plus dépendre de t_1, t_2, par exemple $t_4 : a \times b$ avec un traitement séparé de $-d$.)

	p	T_p	S_p	E_p
Version 1	3	2	2	2/3
Version 2	2	2	2	1

Remarque

Dans les notes, les valeurs de l'efficacité semblaient inversées. Avec $E = S/p$, la version 1 (3 processeurs) donne 2/3 et la version 2 (2 processeurs) donne 1.

4 Parallélisme de flux : exemple

Pour i de 1 à n faire $C(i) \leftarrow (A(i)+2) \times B(i)/3$. On découpe en 3 étapes : ϑ_1 (addition), ϑ_2 (multiplication), ϑ_3 (division), affectées à P_1, P_2, P_3. À chaque pas de temps, la donnée i passe de P_1 à P_2 puis à P_3, pendant que P_1 traite déjà la donnée $i+1$.

Avec une étape par processeur et n données :

$$T_1 = 3n, \qquad T_3 = n+2,$$

$$S_3 = \frac{3n}{n+2} \xrightarrow[n\to\infty]{} 3, \qquad E_3 = \frac{n}{n+2} \xrightarrow[n\to\infty]{} 1.$$

Généralisation (p étapes) :

$$T_1 = p\,n, \qquad T_p = n + p - 1, \qquad S_p = \frac{p\,n}{n+p-1} \simeq p, \qquad E_p = \frac{n}{n+p-1} \simeq 1 \quad \text{si } p \ll n.$$

Les $p-1$ unités de temps supplémentaires correspondent au remplissage (et à la vidange) du pipeline.

5 Grain (granularité) et degré de parallélisme

5.1 Granularité

Définition

Le **grain** (ou granularité) est la taille moyenne des tâches élémentaires issues de la parallélisation. Il se calcule en *temps d'exécution* ou en *nombre d'instructions*.

- Pour diviser le programme en blocs, on prend en compte le nombre de processeurs.
- La granularité dépend de l'architecture :
  - communication **lente** ⇒ granularité **grosse** (peu de communications, tâches longues) ;
  - communication **rapide** ⇒ granularité **fine**.
- Granularité fine → plus de parallélisme (mais plus de surcoût de communication et de synchronisation).

5.2 Degré de parallélisme

Définition

Le **degré de parallélisme** est le nombre de sous-tâches exécutées simultanément dans un programme parallèle. Il peut varier pendant l'exécution.

On considère souvent le degré **maximal** $D_{\max}$, **moyen** D_{moy} et **minimal** $D_{\min}$. Ils peuvent être établis *statiquement* (à la compilation) ou *dynamiquement* (à l'exécution). $D_{\max}$ indique combien de processeurs on peut utiliser au maximum.

Exemple. Programme composé de S_0 ; $S_1 \parallel S_2 \parallel S_3$; $S_4 \parallel S_5 \parallel S_6 \parallel S_7$; $S_8 \parallel S_9$; S_{10}. Les degrés successifs valent $1, 3, 4, 2, 1$:

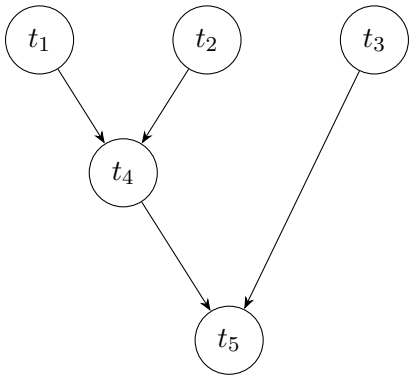
$$D_{\max} = 4, \quad D_{\min} = 1, \quad D_{\text{moy}} = \frac{1+3+4+2+1}{5} = \frac{11}{5} = 2{,}2.$$

6 Temps et nombre de processeurs optimaux

Définitions

- T_{opt} : temps optimal d'exécution parallèle.
- P_{opt} : nombre *minimal* de processeurs permettant d'obtenir T_{opt}.

Graphe de dépendance. C'est un graphe orienté acyclique (DAG) : les sommets sont les tâches et les arcs les dépendances. T_{opt} est la somme des longueurs des tâches constituant le **chemin le plus long** (chemin critique).



Ici le chemin critique ($t_1 \to t_4 \to t_5$) compte 3 tâches, donc $T_{\text{opt}} = 3$.

Exemple (4 tâches). $t_1 : a \leftarrow a + 2$, $t_2 : b \leftarrow b - 4$, $t_3 : c \leftarrow d + 9$, $t_4 : d \leftarrow a \times b - d$, avec $(t_1 \parallel t_2 \parallel t_3) \Rightarrow t_4$:

$$D_{\max} = 3,\ D_{\min} = 1,\ D_{\text{moy}} = 2,\ T_{\text{opt}} = 2,\ P_{\text{opt}} = 3 = D_{\max}.$$

Point important

P_{opt} est un optimum **global** : il ne dépend pas de la représentation du programme. Si une représentation donnée utilise 4 processeurs mais qu'une autre atteint le même T_{opt} avec 3, alors $P_{\text{opt}} = 3$ et non 4.

7 Évaluation des performances

On évalue le parallélisme pour savoir s'il est efficace, au moyen de l'**accélération** et de l'**efficacité**.

Terminologie. $T_1(n)$: temps d'exécution séquentiel (1 processeur) ; $T_p(n)$: temps d'exécution parallèle (p processeurs) ; n : taille du problème.

7.1 Accélération (speedup)

$$S_p(n) = \frac{T_1(n)}{T_p(n)}$$

1. $S_p(n) < 1$: on **ralentit** (mauvaise parallélisation).
2. $1 < S_p(n) \leq p$: cas **normal**.
3. $S_p(n) > p$: accélération **super-linéaire**.

Pourquoi une accélération super-linéaire ? Dans un programme séquentiel, le temps de traitement est : T_1 = temps de swap (échanges mémoire/cache) + temps de traitement. Si les données sont trop grandes pour la mémoire cache d'un seul processeur (par exemple une matrice de $10^6 \times 10^6$ lue ligne par ligne), il y a beaucoup de swaps. Dans le cas parallèle idéal, chaque processeur ne manipule qu'une partie des données, qui tient en cache : le swap est éliminé et

$$T_p = \frac{\text{temps de traitement}}{p} < \frac{T_1}{p},$$

d'où $S_p > p$.

7.2 Efficacité

$$E_p(n) = \frac{S_p(n)}{p} = \frac{T_1(n)}{p\,T_p(n)}$$

Elle mesure le taux d'utilisation des processeurs : $E_p = 1$ est l'idéal (tous les processeurs sont occupés en permanence).

8 Compléments

Loi d'Amdahl

Si une fraction f du programme est strictement séquentielle, alors

$$S_p \leq \frac{1}{f + \frac{1-f}{p}} \xrightarrow[p\to\infty]{} \frac{1}{f}.$$

Par exemple, avec $f = 10\,\%$, l'accélération ne dépassera jamais 10, quel que soit le nombre de processeurs.

- **Compromis de granularité** : grain trop fin ⇒ surcoût de communication et de synchronisation ; grain trop gros ⇒ déséquilibre de charge et processeurs inactifs.
- **Équilibrage de charge** : l'efficacité est maximale quand tous les processeurs ont la même quantité de travail.
- **Lien avec le chemin critique** : on a toujours $T_p \geq T_{\text{opt}}$, donc $S_p \leq T_1/T_{\text{opt}}$, quel que soit p.